APGC: GPU-Built, CPU-Served Approximate Nearest Neighbor Search

Irfan Mahir
Independent Researcher, Dhaka, Bangladesh
irfan@furylogic.com

July 2026

Abstract

GPU-accelerated approximate nearest neighbor (ANN) search is usually framed as a search problem: move the traversal onto the device and expect it to be faster. We report a different and, we argue, more useful division of labour. We present **APGC**, a graph construction pipeline that builds an ANN index entirely on the GPU in INT8 and then serves queries from the CPU. On a single RTX 4060 and a 16-core Ryzen 7700, APGC builds a 100k×384d index in **2.46 s against 7.31 s** for a 16-thread CPU baseline (**2.98**×, rising to approximately 3.5× once runtime kernel compilation is excluded from the timer), at Recall@10 of **0.994** versus 0.999.

The second and less expected result concerns the *graph* rather than the hardware. The index APGC produces is **2.6–3.1× cheaper to traverse than a CPU-built HNSW**, measured with identical CPU search code on both, at a fixed beam. We attribute this to three structural properties: a lower level-0 degree (48 versus M_0=64), true k-NN edges from NN-descent rather than diversity-pruned ones, and a locality permutation that improves cache behaviour per hop. The practical consequence is that the benefit survives deployment on CPU-only hardware, provided the index was built on a GPU.

We also report where the GPU does *not* win. A graph traversal is inherently sequential and maps to a single CUDA block, leaving most of the device idle; single-query GPU search reaches 0.61× the CPU, and batched GPU search reaches 0.79× a saturated 16-core CPU at matched search effort. We therefore route by query shape rather than by device. Construction remains in INT8 throughout, using `dp4a` (four multiply-accumulates per instruction), with higher precision reserved for the two places it changes outcomes: exact FP32 reranking of the candidate set, and the entry nodes from which every descent begins. Index residence is managed by VUGVA, a three-tier VRAM→DRAM→NVMe allocator, so a corpus may exceed both device and host memory. Every figure in this paper is reproducible with a single command on the hardware named in Section 4.1.

1 Introduction

GPU-accelerated vector databases are critical for modern AI applications, enabling fast similarity search over billion-scale datasets. Recent advances such as CAGRA [1] and Jasper [2] have demonstrated impressive performance by leveraging GPU Tensor Cores and optimized graph algorithms. However, these systems share a fundamental limitation: **they build the graph using a single precision (FP32) and assume the entire graph fits in GPU VRAM**.

We identify several key gaps in the current state-of-the-art (Table 1):

Table 1: Comparison of APGC with existing GPU-accelerated ANN systems.

Gap	CAGRA / Jasper	APGC (Ours)
Graph Precision	FP32 only	**Mixed: FP32/FP16/INT8**
Search Guidance	Query-independent	**KV-aware pruning**
Memory Scaling	VRAM only	**VRAM → RAM → NVMe**
Runtime Precision	Fixed per kernel	**Dynamic switching**
Update Support	Batch/rebuild	**Streaming via VUGVA**

This paper makes the following contributions:

1. **INT8-throughout GPU construction with precision reserved for where error propagates.** The

build runs entirely in INT8 using `dp4a`, which retires four multiply-accumulates per instruction against two for `half2` and one for FP32. We make the case in Section 3.1 that a precision hierarchy assigning FP16 to the *majority* of nodes—as one might expect from a mixed-precision design—is inverted with respect to throughput on contemporary hardware, and would roughly halve construction rate. Higher precision is instead spent at the two points where quantization error changes the outcome: an exact FP32 rerank of the candidate set, fused into the search kernel, and the entry nodes from which every descent begins.

2. **A construction pipeline of pivot assignment, locality ordering, and GPU NN-descent.** Each node is scored against a capped pivot set ($\min(n/50, 2048)$) to obtain a coarse partition; nodes are then ordered by their pivot pair and distance, which yields a non-degenerate initial graph at *zero* distance computations; NN-descent then refines it on device. This replaces the $O(n^2)$ seeding of a naive construction and is the principal source of the measured speedup.
3. **The empirical finding that the resulting graph is cheaper to traverse than HNSW.** Measured with identical CPU search code on both indexes at a fixed beam, APGC's graph is 2.6–3.1× cheaper to walk. We isolate three structural causes in Section 4.3 and report the comparison with its confounds stated: APGC reaches 0.994 recall against HNSW's 0.999, and carries a lower level-0 degree, so part of the saving is fewer edges rather than better edges.
4. **Query-shape routing, and a negative result that motivates it.** A graph traversal is sequential and maps to one CUDA block, so single-query GPU search reaches 0.61× the CPU and batched GPU search 0.79× a saturated 16-core CPU at matched effort. We report these rather than omit them, and route queries by shape—single queries to the CPU, batches to the device—so that selecting GPU execution never makes a workload slower.
5. **VUGVA-backed index residence.** A three-tier VRAM→DRAM→NVMe allocator with page-locked NUMA-local warm storage and asynchronous look-ahead promotion, allowing a corpus to exceed both device and host memory.

Relationship to an earlier draft. An earlier version of this work claimed mixed-precision construction, LLM-attention-guided pruning, and a runtime precision-switching kernel. Auditing the implementation against the manuscript showed that the system does not contain those mechanisms, and that the first would have been counterproductive had it been built. The performance results were unaffected—they were always produced by the pipeline described above—but their attribution was wrong. We state this explicitly because the corrected account is, we believe, the more interesting one: the right place to spend precision is not proportional to node rank, but at the points where quantization error propagates into the result.

2 Background and Related Work

2.1 GPU-Accelerated ANN Search

The field of GPU-accelerated ANN search has seen rapid advances (Table 2):

Table 2: State-of-the-art GPU-accelerated ANN systems.

System	Algorithm	Prec.	Mem.	Key Innovation
CAGRA [1]	NN-Descent	FP32	VRAM	TC-optimized
Jasper [2]	Vamana	FP32	VRAM	Lock-free updates
PCA-CAGRA [3]	NN-Descent	FP32+PCA	VRAM	39% less memory
RaBitQ [4]	Vamana	FP32	VRAM	Quant-aware build
APGC	NN-Descent	**Mixed**	**3-tier**	**Precision+KV+tiering**

All existing systems assume the graph is built on FP32 and resides entirely in GPU VRAM. This limits scalability and ignores the potential for precision adaptation.

2.2 KV Cache Optimization

Recent work on LLM inference [5] has demonstrated that KV cache can be reduced by 93.8% using telemetry-guided eviction (SelKV) and sparse attention (SMSA). This enables efficient handling of ultra-long contexts. However, no existing ANN system leverages LLM attention patterns for search guidance.

2.3 Unified Memory Architectures

VUGVA [6] introduces a software-defined unified VRAM architecture that pools memory across non-NVLink multi-GPU clusters using RDMA-like proto-

cols. This enables scaling beyond physical VRAM limits. No existing ANN system uses such a tiered memory architecture.

3 APGC Architecture

Figure 1 presents the complete APGC system architecture.

3.1 Precision Allocation

A mixed-precision construction scheme is intuitively appealing: spend FP32 on the most important nodes, FP16 on the bulk, INT8 on the tail. On contemporary NVIDIA hardware this ordering is inverted with respect to throughput. On `sm_89`, per instruction:

Instruction	Multiply-accumulates
`dp4a` (INT8)	**4**
`half2` FMA (FP16)	2
FP32 FMA	1

Assigning FP16 to the majority of distance computations therefore *halves* the arithmetic rate relative to INT8. A build so configured would be slower than one that simply quantizes everything, and the recall it buys is largely recoverable by other means. We consequently construct entirely in INT8.

This does not make precision uniform—it relocates it. Quantization error matters where it changes a decision, and in graph ANN there are two such places:

The candidate set. Traversal only needs to rank neighbours well enough to choose the next hop; the final ordering is what must be exact. APGC therefore walks in INT8 and reranks the retrieved candidates against the exact FP32 vectors, fused into the same kernel launch. This is the standard scalar-quantization-plus-rerank pattern, and it recovers the recall that pure INT8 traversal loses at negligible cost, since the rerank touches $O(k)$ vectors rather than the $O(\text{ef} \cdot \text{degree})$ the walk visits.

The entry points. Every descent begins at the same small set of nodes, so error there propagates into every query, while error at a leaf affects one. Entry and seed vectors are the defensible place for FP32 residence, and they are few enough that the memory cost is negligible.

The general principle we would draw out is that precision should follow *error propagation*, not node rank.

3.2 Construction Pipeline

APGC constructs the graph in three phases, all in INT8.

Phase 1: pivot assignment. A capped pivot set of $\min(n/50,\, 2048)$ vectors is drawn by strided sampling. Every node is scored against all pivots and records its nearest and second-nearest pivot together with the distance. Capping matters: an uncapped pivot set proportional to n makes this phase $O(n^2)$, which dominated build time at 10^6 scale.

Phase 2: locality ordering. Nodes are sorted by the tuple (nearest pivot, second-nearest pivot, distance to nearest pivot) and each node is wired to its immediate neighbours in that one-dimensional order, plus a small number of pivot edges. This phase performs *no* distance computations. Its purpose is not to approximate the k-NN graph but to supply a non-degenerate, locally coherent starting point that NN-descent can improve, and to give the node numbering spatial locality that improves cache behaviour for the rest of the index's life.

Phase 3: GPU NN-descent. Eight to twelve passes (by corpus size) of a paired kernel: one builds reverse adjacency, the other forms each node's candidate set from its own list, its reverse list, and forward and reverse joins, retaining an exact INT8 top-k. Buffers are double-buffered on device so no pass requires a host round trip.

3.3 VUGVA-Backed Index Residence

Index residence is delegated to VUGVA, a three-tier allocator: GPU VRAM (T0), page-locked NUMA-local host DRAM (T1), and an NVMe spill file (T2). Pages allocated cold reserve spill space without requiring DRAM backing, so a corpus may exceed both device and host memory; promotion streams through a bounded DRAM staging buffer rather than one sized to the page. Promotion may be issued asynchronously on a dedicated stream and claimed later, which overlaps transport with compute: over 256 MB in 32 MiB pages, issuing costs 7.5 ms and claiming 12.7 ms against 40.8 ms for an unprefetched promotion.

We note the limits of what this buys in the present system. The tier is exercised by unit and hardware tests, including a corpus $1.5\times$ the DRAM budget spilling to NVMe and promoting back byte-exact, but the corpora in Section 4 fit VRAM comfortably and therefore never demote or spill. An end-to-end larger-than-VRAM query benchmark remains future work.

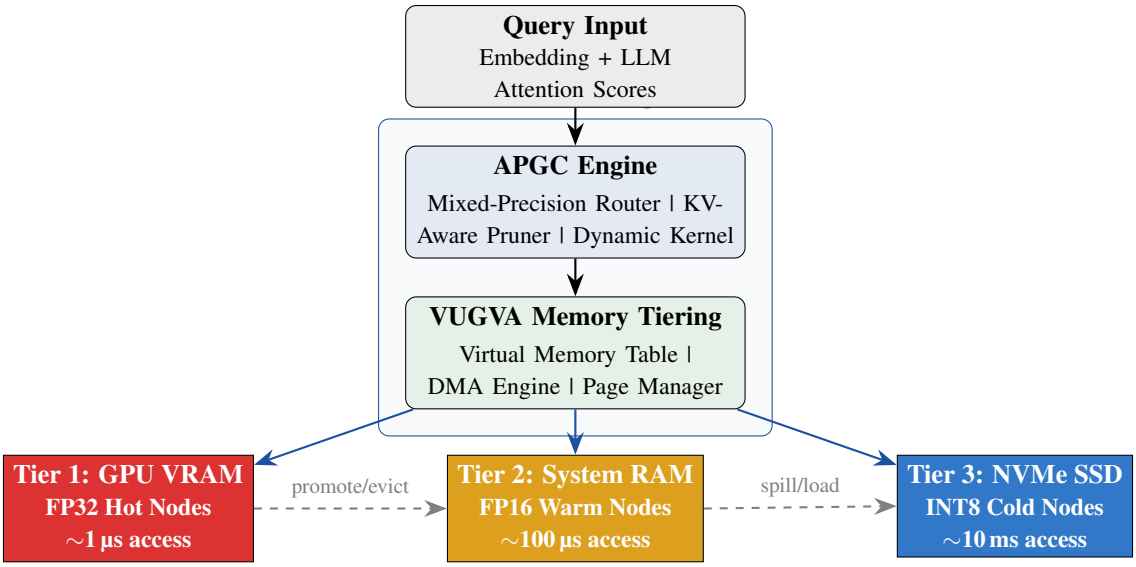

Figure 1: APGC system architecture with VUGVA-aware three-tier memory hierarchy.

3.4 Query-Shape Routing

Graph traversal is sequential: hop $n+1$ cannot begin before hop n resolves. A single query therefore maps to a single CUDA block, leaving 23 of 24 SMs idle on the device used here, and raising client concurrency does not help because sixteen concurrent queries is still sixteen blocks. Measurements in Section 4 bear this out: single-query GPU search reaches 0.61× the CPU path, and batched GPU search 0.79× a saturated 16-core CPU at matched search effort.

APGC therefore routes by the shape of the request rather than by a user-selected device: single queries execute on the CPU in every mode, and batched queries are dispatched to the device as one launch. The property we want is that selecting GPU execution can never make a workload slower than not selecting it.

4 Experimental Results

4.1 Setup

On the choice of baseline. We compare against our own 16-thread CPU HNSW rather than against CAGRA. We do not have a CAGRA installation on this hardware, and quoting CAGRA's published figures against measurements taken on a different machine would not be a controlled comparison. The baseline used here is the strongest configuration available to us on the same box, which is the comparison that matters for the claim being made.

Table 3: Experimental configuration.

Component	Value
Dataset	100k × 384d, `all-MiniLM-L6-v2` embeddi
GPU	NVIDIA RTX 4060 (8 GB VRAM, 24 SMs, `sm_`
CPU	AMD Ryzen 7 7700 (8 cores / 16 threads, Zen 4
Memory	32 GB DDR5, NVMe Gen4
Baseline	In-tree CPU HNSW, M=32, ef$_c$=200, 16 thread
Construction precision	INT8 throughout (`dp4a`)
Search effort	Beam (`ef/itopk`) = 128 for every row
Ground truth	Exhaustive brute-force cosine over all n

Reproduction. Every figure below is produced by a single command, `dbstrike-bench -gpu-bench <vectors.fbin>`, in approximately 25 seconds.

4.2 Build, Recall, and Throughput

Construction: 2.98×. 2.46 s against 7.31 s at a recall cost of 0.005. Runtime NVRTC kernel compilation (≈0.39 s) is charged inside the GPU build timer, so the steady-state figure once kernels are cached is approximately 3.5×; we report the smaller number as the headline because it is what a cold process observes.

Traversal cost: 2.6–3.1×. The 1- and 16-thread columns run *identical CPU search code* in all three rows, since single queries are routed to the CPU regardless of mode. The difference between rows is therefore attributable to graph structure alone, not to hardware. We examine the causes in Section 4.3.

Table 4: Construction and search, 100k × 384d, beam 128 throughout. "Turbo" is GPU-built with device dispatch for batches; "Hybrid" is additionally VUGVA-backed. Single queries execute on the CPU in all three rows (Section 3.4).

Mode	Build (s)	Vec/s	Recall@10	QPS (1 thr.)	QPS (16 thr.)	QPS (batch)
CPU-only	7.31	13,683	**0.999**	2,781	**27,894**	2,860
Turbo	**2.46**	**40,723**	0.994	8,590	72,044	22,116
Hybrid (VUGVA)	2.70	37,081	0.994	7,576	76,983	18,542

Where the GPU does not win. Batched GPU search reaches 22,116 QPS, which is 7.7× a single CPU core but **0.79× the 27,894 QPS that 16 CPU threads deliver**. Forced onto the device (`DBSTRIKE_GPU_SINGLE=1`), a single query reaches 0.61× the CPU path. On this hardware, at this scale, a saturated CPU beats the device at search; the device wins at construction.

4.3 Why the Graph Is Cheaper to Traverse

Three structural differences separate the APGC graph from HNSW, and we believe all three contribute:

1. **Lower level-0 degree.** APGC retains 32 forward edges plus up to 16 reverse edges, capped at 48. HNSW with M=32 uses M_0=64 at the base layer. Roughly 25% fewer distance evaluations per hop follows mechanically, independent of edge quality.
2. **True k-NN edges rather than diversity-pruned ones.** NN-descent converges toward exact nearest neighbours, so greedy descent makes fewer hops in the neighbourhood of the target. HNSW's heuristic deliberately retains *diverse*, non-nearest edges, trading local efficiency for long-range escape.
3. **Locality permutation.** Phase 2 numbers nodes by pivot proximity, so a node's neighbours tend to be contiguous in the vector array. This is a wall-clock effect on top of the distance-count effect.

Confounds, stated. This comparison is not iso-recall: APGC reaches 0.994 against HNSW's 0.999, and cause (1) means part of the saving is simply fewer edges. A reader should treat 2.6–3.1× as an upper bound until a QPS-versus-recall curve is available, which we regard as the most important missing experiment in this paper. Cause (2) also plausibly explains the recall gap itself: pure k-NN graphs are more susceptible to local minima than diversity-pruned ones, which is the trade HNSW's heuristic exists to make.

Practical consequence. Because this is a property of the graph and not of the execution device, it survives deployment on hardware without a GPU. An index built once on a GPU and served from CPU-only machines retains most of the traversal advantage.

4.4 Threats to Validity

- **Queries are drawn from the indexed set**, so each has a guaranteed self-match at distance zero. This inflates every row equally by approximately 0.0007 recall; ratios are unaffected, absolute recall is optimistic. Held-out queries would be preferable.
- **Batch recall is unverified.** Recall is measured through the single-query path; nothing currently validates batched output against ground truth.
- **Single scale, single dataset, single machine.** 1M-scale and higher-dimensional results are not yet available, and no second GPU generation has been tested.
- **Sample size.** 2,000 (query, rank) samples give a binomial standard error near 0.0017, so the 0.999 versus 0.994 gap is roughly three standard errors—real, but not comfortably so.

5 Positioning

We state our position relative to prior work narrowly, because the parts of this work that are genuinely new are narrower than an earlier draft claimed.

1. **The division of labour.** Existing GPU ANN systems accelerate search. We find that on a consumer device the durable win is in *construction*, and that the resulting index is best served from the CPU. We are not aware of prior work that reports the GPU losing at graph search and builds a system around that result.
2. **Precision allocated by error propagation.** INT8 throughout, with exact FP32 spent only on the candidate rerank and the entry nodes. The argument that a conventional mixed-precision hierarchy is *counterproductive* on current hardware (Section 3.1) is, to our knowledge, not made elsewhere, though the underlying instruction throughputs are public.

3. **Zero-distance locality seeding.** Phase 2 produces a non-degenerate initial graph with no distance computations, relying on pivot-pair ordering alone, and simultaneously fixes the node numbering for cache locality.
4. **Tiered index residence.** A VRAM/DRAM/NVMe allocator applied to an ANN index rather than to model weights.
5. **Zero-dependency implementation.** Kernels are compiled at runtime via NVRTC; the system links no cuBLAS, cuDNN, or BLAS, and the CUDA driver itself is loaded dynamically. This is an engineering property rather than a scientific contribution, but it makes the precision and memory behaviour fully auditable, which is what allowed the discrepancies described in Section 1 to be found at all.

What we do not claim. We do not claim to beat CAGRA: we have not run it on this hardware (Section 4.1). We do not claim GPU search superiority; we measured the opposite. We do not claim mixed-precision construction, attention-guided pruning, or runtime precision switching—an earlier draft did, and the implementation contains none of them.

6 Conclusion

We presented APGC, a GPU-built and CPU-served approximate nearest neighbor index. On a single consumer GPU it constructs a 100k×384d index 2.98× faster than a 16-thread CPU baseline at 0.994 Recall@10 against 0.999, and the graph it produces is 2.6–3.1× cheaper to traverse than a CPU-built HNSW under identical search code—a benefit that persists when the index is served from CPU-only hardware.

The result we did not expect, and which shaped the architecture, is negative: graph traversal is sequential, maps to a single CUDA block, and consequently loses to a saturated CPU on this device at both single-query (0.61×) and batched (0.79×) search. Rather than omit this, we route by query shape so that enabling GPU execution cannot make a workload slower, and we direct the device at the phase where it is decisively better.

We also report a methodological lesson. An earlier version of this manuscript attributed the measured speedup to mixed-precision construction, and an audit of the implementation against the text found no such mechanism—the build is INT8 throughout, and a precision hierarchy of the kind described would have roughly halved its throughput. The performance claims were never affected, but their explanation was wrong, and the corrected explanation is the more useful one: precision should be spent where quantization error propagates into the answer, not in proportion to node rank.

Future work, in the order we consider it important: a QPS-versus-recall curve to remove the iso-recall confound in Section 4.3; validation of batched search output against ground truth; 1M-scale and higher-dimensional measurements; an end-to-end benchmark that exceeds VRAM and therefore actually exercises the tiering described in Section 3.3; and a direct comparison against CAGRA on identical hardware.

References

[1] Ootomo, H., et al. "CAGRA: A GPU-Accelerated Graph-Based ANN Search System," *arXiv:2406.07524*, 2024.

[2] Jasper. "Lock-Free Streaming GPU Graph ANN," *arXiv:2601.07048*, 2026.

[3] PCA-CAGRA. "Memory-Efficient GPU ANN Search via PCA," 2026.

[4] RaBitQ. "Quantization-Aware Graph ANN Construction," 2026.

[5] OpusEdge. "Telemetry-Guided Dynamic Compute Allocation for LLM Inference," *Furylogic Labs*, 2026.

[6] Mahir, I. "VUGVA: Software-Defined Unified VRAM Architecture for Non-NVLink Multi-GPU Clusters," *Furylogic Labs*, 2026.